Blindspot Security Assessment

Find what others miss. - Static APK security assessment report

Developed by @shuuubhraj

APP NAME	PACKAGE	VERSION
DivaApplication.apk	jakhar.aseem.diva	1.0 (1)
SDK (MIN / TARGET)	**FILE**	**SIZE**
15 / 23	DivaApplication.apk	1.4 MB
SHA-256	**ASSESSED**	**STATUS**
5cefc51fce9bd760b92ab2340477f4dda84b4ae0c5d04a8c9493e4fe34fab7c5	2026-10-18T09:03:43.443821	completed

Executive summary

Blindspot performed an automated **static** security assessment of **DivaApplication.apk** (**jakhar.aseem.diva**). The scan produced **32** finding(s): 0 critical, 3 high, 23 medium, 2 low, 4 informational. Critical and high findings should be triaged first; each includes evidence, impact, and remediation guidance below.

Contents

- Scope and methodology
- APK metadata
- Scan coverage
- Severity distribution
- OWASP Mobile Top 10 (2024) mapping
- AI review
- Detailed findings
- Limitations and exclusions

Scope and methodology

This assessment is **static analysis only**: the APK was unpacked and inspected without executing it. No dynamic testing, network interaction, or backend testing was performed.

- APK structure validation, SHA-256 hashing, safe extraction
- AndroidManifest.xml decoding and configuration analysis
- Permission, component-export and intent-filter analysis
- Decompiled-source review (where a decompiler was available)
- MobSFScan static analysis (when available)
- 121 custom Blindspot detection-rule executions across 7 categories
- Optional local-AI review (Ollama, on-device only - never a cloud service)

Severity reflects potential impact if the issue is real and exploitable. Confidence reflects how strongly the evidence supports the finding. The two are independent.

APK metadata

File name	DivaApplication.apk
SHA-256	`5cefc51fce9bd760b92ab2340477f4dda84b4ae0c5d04a8c9493e4fe34fab7c5`
File size	1.4 MB
Package	jakhar.aseem.diva
App name	-
Version	1.0 (1)
SDK (min / target)	15 / 23

Scan coverage

STAGE	STATUS	DURATION	NOTE
`validate`	completed	188.3 ms	-
`extract`	completed	1240.9 ms	extracted 390 entries
`metadata`	completed	142.0 ms	-
`decompile`	completed	57747.8 ms	jadx decompiled 651 java files; apktool decoded resources and smali
`mobsfscan`	completed	2929.9 ms	-
`custom_rules`	completed	83610.0 ms	-
`ai_review`	completed	1866365.2 ms	model=qwen2.5-coder:7b, accepted=10, libs_skipped=629
`normalize`	completed	57.9 ms	-
`finalize`	completed	84.4 ms	-

Rule executions by outcome: `completed`: 1 `finding`: 17 `pass`: 91 `needs_review`: 8 `not_applicable`: 4

Severity distribution

0	3	23	2	4
CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL

OWASP Mobile Top 10 (2024) mapping

CATEGORY	NAME	FINDINGS
`M1`	Improper Credential Usage	4
`M2`	Inadequate Supply Chain Security	0
`M3`	Insecure Authentication/Authorization	2
`M4`	Insufficient Input/Output Validation	7
`M5`	Insecure Communication	4
`M6`	Inadequate Privacy Controls	1
`M7`	Insufficient Binary Protections	2

M8	Security Misconfiguration	6
M9	Insecure Data Storage	5
M10	Insufficient Cryptography	0

OWASP categories are classification metadata, not severity scores.

AI review

Model	qwen2.5-coder:7b
Status	completed
Files reviewed	11
Findings proposed / accepted	11 / 10
Note	1 of 12 file(s) could not be reviewed; scan continues with the rest

AI findings are hypotheses with hallucination controls: every AI-proposed finding is validated against real code locations before it is kept, and findings the static engines independently confirm are marked *ai-corroborated*. AI output never overrides a static-engine result.

#	FINDING	SEVERITY	CONFIDENCE	LOCATION	STATUS
1	WebView JavaScript enabled	medium	high	jadx\sources\jakhar\aseem\diva\InputValidation2URISchemeActivity.java:18	ai-corroborated
2	SQL Injection Vulnerability	medium	low	jadx\sources\jakhar\aseem\diva\NotesProvider.java:48	ai-evidence-supported
3	Hardcoded Sensitive Information	medium	high	jadx\sources\jakhar\aseem\diva\APICreds2Activity.java:14	ai-evidence-supported
4	Insecure Data Storage	medium	high	jadx\sources\jakhar\aseem\diva\InsecureDataStorage1Activity.java	ai-unverified
5	SQL Injection Vulnerability	medium	high	jadx\sources\jakhar\aseem\diva\InsecureDataStorage2Activity.java:24	ai-evidence-supported
6	SQL Injection Vulnerability	medium	high	jadx\sources\jakhar\aseem\diva\SQLInjectionActivity.java:26	ai-evidence-supported
7	Hardcoded Sensitive Information	medium	high	jadx\sources\jakhar\aseem\diva\APICredsActivity.java:10	ai-evidence-supported
8	Hardcoded Secret Key	medium	high	jadx\sources\jakhar\aseem\diva\HardcodeActivity.java:14	ai-evidence-supported
9	Insecure Storage of Sensitive Data	medium	high	jadx\sources\jakhar\aseem\diva\AccessControl3Activity.java:17	ai-evidence-supported
10	Insecure Data Storage	medium	high	jadx\sources\jakhar\aseem\diva\InsecureDataStorage4Activity.java	ai-unverified

Detailed findings

1. Application is debuggable

HIGH CONFIDENCE: HIGH STATIC

94b22f97-0001 · rule BS-M01 v1.0.0 · engine: custom · manifest

Location	AndroidManifest.xml
OWASP	M8
CWE	CWE-489

Description

Application tag sets android:debuggable="true". The app can be attached by a debugger on any device with adb access.

Evidence

```
application tag parsed with debuggable=true
```

Severity rationale: HIGH: a debuggable release build exposes the running process to debugger attachment on user devices, enabling memory inspection, secret extraction, and bypass of local protections.

Confidence rationale: HIGH: the flag is a literal boolean on the application tag, parsed directly from the decoded manifest.

Remediation

Remove android:debuggable="true" from the release manifest (or set it to false). Keep debuggable builds internal only; verify the shipped APK with `aapt dump badging` / apkanalyzer and in CI.

2. Exported content provider without read/write permission

HIGH CONFIDENCE: HIGH STATIC

94b22f97-0005 · rule BS-M10 v1.0.0 · engine: custom · manifest

Location	AndroidManifest.xml
OWASP	M8
CWE	CWE-926

Description

1 exported provider(s) with no readPermission/writePermission: jakhar.aseem.diva.NotesProvider.

Evidence

```
android:exported="true" on <provider android:name="jakhar.aseem.diva.NotesProvider"
android:authorities="jakhar.aseem.diva.provider.notesprovider"> (no readPermission/writePermission)
```

Severity rationale: HIGH: an unguarded exported provider exposes its backing data to arbitrary apps for read and write, enabling theft and tampering of app data.

Confidence rationale: HIGH: exported state, authorities, and the absence of read/write permission attributes come straight from the parsed provider declarations.

Remediation

Set android:exported="false" for providers that are app-internal. For providers that must be shared, set android:readPermission and android:writePermission (signature level where possible), use path-permissions for granularity, and validate all query parameters.

3. SQL injection via concatenated query

HIGH CONFIDENCE: MEDIUM STATIC

94b22f97-0018 · rule BS-W08 v1.0.0 · engine: custom · webview

Location	jadx\sources\jakhar\aseem\diva\InsecureDataStorage2Activity.java : lines 31
OWASP	M4
CWE	CWE-89

Description

SQL statement built by concatenating variables (2 call site(s)).

Evidence

```
       29 |         EditText pwd = (EditText) findViewById(R.id.ids2Pwd);
       30 |         try {
>>>    31 |             this.mDB.execSQL("INSERT INTO myuser VALUES ('" + usr.getText().toString() + "', '" +
pwd.getText().toString() + "');");
       32 |             this.mDB.close();
       33 |         } catch (Exception e) {

       33 |         EditText srchtxt = (EditText) findViewById(R.id.ivi1search);
       34 |         try {
>>>    35 |             Cursor cr = this.mDB.rawQuery("SELECT * FROM sqliuser WHERE user = '" +
srchtxt.getText().toString() + "'", null);
       36 |             StringBuilder strb = new StringBuilder("");
       37 |             if (cr != null && cr.getCount() > 0) {
```

Severity rationale: HIGH: SQL injection in the local database can exfiltrate or corrupt app data; concatenated values are the classic injection pattern.

Confidence rationale: MEDIUM: concatenation of variables into the SQL string is directly observed, but whether any variable is attacker-controlled (vs. app-internal) is not proven; exploitability not confirmed.

Remediation

Use parameterized queries with ? placeholders and selectionArgs / bind arguments. Never concatenate values into SQL strings.

4. Token used without expiry or refresh handling

MEDIUM CONFIDENCE: LOW HEURISTIC

94b22f97-0019 · rule BS-A05 v1.0.0 · engine: custom · auth

Location	jadx\sources\android\support\v7\app\NotificationCompat.java : lines 18
OWASP	M3
CWE	CWE-613

Description

5 file(s) read auth/session tokens with no expiry, refresh, or deadline handling visible in the same file. Static analysis cannot confirm whether the server enforces token expiry or rotates refresh tokens; review the token lifecycle server-side before treating this as a vulnerability.

Evidence

```
       16 |         if (style instanceof MediaStyle) {
       17 |             MediaStyle mediaStyle = (MediaStyle) style;
>>>    18 |             NotificationCompatImpl21.addMediaStyle(builder, mediaStyle.mActionsToShowInCompact,
mediaStyle.mToken != null ? mediaStyle.mToken.getToken() : null);
       19 |         }
       20 |     }

       31 |         public void notify(String packageName, int id, String tag, Notification notification) throws
RemoteException {
       32 |             NotificationCompatSideChannelService.this.checkPermission(getCallingUid(), packageName);
>>>    33 |             long idToken = clearCallingIdentity();
       34 |             try {
       35 |                 NotificationCompatSideChannelService.this.notify(packageName, id, tag, notification);

       78 |     }
       79 |
>>>    80 |     public MediaControllerCompat(Context context, MediaSessionCompat.Token sessionToken) throws
RemoteException {
       81 |         if (sessionToken == null) {
       82 |             throw new IllegalArgumentException("sessionToken must not be null");

       32 |     }
       33 |
>>>    34 |     public static Object fromToken(Context context, Object sessionToken) {
       35 |         return new MediaController(context, (MediaSession.Token) sessionToken);
       36 |     }

      450 |         }
      451 |
>>>   452 |         public Object getToken() {
      453 |             return this.mInner;
      454 |         }
```

Severity rationale: Non-expiring tokens extend the replay window after theft (Medium), but only if the server also fails to enforce expiry, which is unverified.

Confidence rationale: No expiry-related check was found near token usage in the client, but token lifetime policy is a server-side property that static analysis cannot observe.

Remediation

Issue short-lived access tokens with explicit expiry, validate exp on every use, and implement refresh-token rotation. Enforce expiry server-side; client-side checks are defense in depth, not the control.

5. Authentication secrets written to logs or stack traces

MEDIUM CONFIDENCE: HIGH STATIC

94b22f97-0020 · rule BS-A06 v1.0.0 · engine: custom · auth

Location	`jadx\sources\android\support\v4\media\MediaBrowserCompat.java` : lines 506
OWASP	M3
CWE	CWE-532

Description

1 logging call(s) include password/token/session keywords, leaking authentication secrets to logcat, bug reports, or crash dumps.

Evidence

```
      504 |             Log.d(TAG, "  mServiceCallbacks=" + this.mServiceCallbacks);
      505 |             Log.d(TAG, "  mRootId=" + this.mRootId);
>>>   506 |             Log.d(TAG, "  mMediaSessionToken=" + this.mMediaSessionToken);
      507 |         }
      508 |
```

Severity rationale: Leaked tokens/passwords enable direct impersonation, but exploitation needs log access, which is restricted on modern Android (Medium).

Confidence rationale: The logging call with the secret keyword is present verbatim at the cited line; the leak is deterministic.

Remediation

Never log passwords, tokens, or session identifiers. Log only redacted placeholders (e.g. token prefix + "..."). Strip verbose logging from release builds with ProGuard/R8 or BuildConfig-gated logging wrappers.

6. Application backup enabled

MEDIUM CONFIDENCE: HIGH STATIC

94b22f97-0002 · rule BS-M02 v1.0.0 · engine: custom · manifest

Location	`AndroidManifest.xml`
OWASP	M9
CWE	CWE-312

Description

android:allowBackup is true (defaulted (attribute absent)). App-private data can be extracted via adb backup / Auto Backup and restored elsewhere.

Evidence

```
android:allowBackup attribute absent; Android defaults it to true
```

Severity rationale: MEDIUM: backup exposes app-private storage to anyone with adb access; impact depends on what sensitive data the app stores, which cannot be fully determined from the manifest alone.

Confidence rationale: HIGH: allowBackup is a manifest-level boolean; when the attribute is absent Android defaults it to true, which is documented platform behavior.

Remediation

Set android:allowBackup="false" unless backup is a genuine product requirement. If backup is required, use a restrictive android:fullBackupContent / android:dataExtractionRules file that excludes sensitive files, and encrypt sensitive data at rest.

7. Outdated SDK policy (targetSdk / minSdk)

MEDIUM CONFIDENCE: HIGH STATIC

94b22f97-0003 · rule BS-M04 v1.0.0 · engine: custom · manifest

Location	`AndroidManifest.xml`
OWASP	M8

Description

Weak SDK policy: targetSdkVersion=23 is below the recommended minimum of 30; minSdkVersion=15 predates the runtime-permission model (API 23): dangerous permissions are granted at install time.

Evidence

```
android:targetSdkVersion="23"

android:minSdkVersion="15"
```

Severity rationale: MEDIUM: an outdated targetSdk forfeits platform hardening applied per API level; minSdk < 23 additionally removes runtime permission control on old devices. Both widen the attack surface but do not directly disclose data by themselves.

Confidence rationale: HIGH: SDK versions are literal integers from the manifest's <uses-sdk> declaration.

Remediation

Raise targetSdkVersion to 30 or higher (ideally the latest) and test against the new behavior changes. Raise minSdkVersion to 23+ so the runtime permission model applies on all supported devices.

8. Intent-filter-reachable component without permission guard

MEDIUM CONFIDENCE: MEDIUM HEURISTIC

94b22f97-0006 · rule BS-M14 v1.0.0 · engine: custom · manifest

Location	`AndroidManifest.xml`
OWASP	M8

Description

Components reachable via intent filters with no permission guard (review whether exposure is intentional): jakhar.aseem.diva.MainActivity <activity> (implicitly exported (intent-filter, targetSdk < 31)); jakhar.aseem.diva.APICredsActivity <activity> (implicitly exported (intent-filter, targetSdk < 31)); jakhar.aseem.diva.APICreds2Activity <activity> (implicitly exported (intent-filter, targetSdk < 31)).

Evidence

```
<activity android:name="jakhar.aseem.diva.MainActivity"> declares 1 intent-filter(s), no permission

<activity android:name="jakhar.aseem.diva.APICredsActivity"> declares 1 intent-filter(s), no permission

<activity android:name="jakhar.aseem.diva.APICreds2Activity"> declares 1 intent-filter(s), no permission
```

Severity rationale: MEDIUM: intent-filter reachability without any permission check is the standard setup for component-hijacking issues; severity depends on what the component does with attacker input, which needs manual review.

Confidence rationale: MEDIUM: reachability is certain from the intent filters, but whether the exposure is a vulnerability or intended behavior (share target, deep link) cannot be decided statically.

Remediation

For each listed component decide: if external access is not needed, set android:exported="false"; if it is needed, add a permission guard (signature level where possible) and validate all incoming intents. Record the decision so the exposure is intentional, not accidental.

9. Cleartext traffic allowed by platform default

MEDIUM CONFIDENCE: HIGH STATIC

94b22f97-0008 · rule BS-M17 v1.0.0 · engine: custom · manifest

Location	`AndroidManifest.xml`
OWASP	M5
CWE	CWE-319

Description

android:usesCleartextTraffic is unset and targetSdkVersion=23 (< 28), so the platform default permits cleartext HTTP traffic.

Evidence

```
android:usesCleartextTraffic attribute absent; android:targetSdkVersion="23"
```

Severity rationale: MEDIUM: cleartext HTTP may be used without any explicit opt-in, exposing data to passive observers; MEDIUM rather than HIGH because no explicit cleartext opt-out was made and actual HTTP usage is unconfirmed.

Confidence rationale: HIGH: the attribute's absence and the targetSdk value are directly observed, and the pre-28 default-allow behavior is documented platform behavior.

Remediation

Set android:usesCleartextTraffic="false" explicitly and raise targetSdkVersion to 28+. Migrate all endpoints to HTTPS; scope any unavoidable HTTP exception to a single host via a network security config.

10. Implicitly exported component (missing android:exported)

MEDIUM CONFIDENCE: LOW HEURISTIC

94b22f97-0009 · rule BS-M18 v1.0.0 · engine: custom · manifest

Location	`AndroidManifest.xml`
OWASP	M8
CWE	CWE-926

Description

Decoded manifest XML text is unavailable, so the presence of android:exported attributes cannot be verified for 3 intent-filter component(s) on targetSdk=23.

Evidence

```
<activity android:name="jakhar.aseem.diva.MainActivity">

<activity android:name="jakhar.aseem.diva.APICredsActivity">

<activity android:name="jakhar.aseem.diva.APICreds2Activity">
```

Severity rationale: MEDIUM: if these components lack android:exported on targetSdk < 31 they are implicitly exported.

Confidence rationale: LOW: without the manifest XML text the android:exported attribute cannot actually be inspected.

Remediation

Add an explicit android:exported="true" or "false" to every component (required for targetSdk 31+ anyway). Set it to false unless external access is genuinely required, and guard exported components with a permission.

11. Content provider uses unsanitized URI input

MEDIUM CONFIDENCE: LOW HEURISTIC

94b22f97-0021 · rule BS-P07 v1.0.0 · engine: custom · platform

Location	`jadx\sources\jakhar\aseem\diva\NotesProvider.java` : lines 69
OWASP	M4
CWE	CWE-20

Description

1 provider(s) consume URI path/query input without visible validation or UriMatcher routing.

Evidence

```
       67 |                 break;
       68 |             case 2:
>>>    69 |                 String id = uri.getLastPathSegment();
       70 |                 count = this.mDB.delete(TABLE, "_id = " + id + (!TextUtils.isEmpty(selection) ? " AND
(" + selection + ')' : ""), selectionArgs);
       71 |                 break;
```

Severity rationale: Medium: unsanitized provider input can reach file or SQL operations, enabling traversal or injection.

Confidence rationale: Low: usage of URI accessors is confirmed, but whether the values reach a sink unvalidated needs data-flow review.

Remediation

Route all provider URIs through UriMatcher, validate path segments against an allowlist, reject ".." and absolute paths, use parameterized queries, and never concatenate URI input into file paths or SQL.

12. Native library loaded from variable or writable location

MEDIUM CONFIDENCE: LOW HEURISTIC

94b22f97-0023 · rule BS-P10 v1.0.0 · engine: custom · platform

Location	jadx\sources\jakhar\aseem\diva\DivaJni.java : lines 12
OWASP	M7
CWE	CWE-494

Description

1 native-load call(s) use a variable library name or a writable-location path.

Evidence

```
      10 |
      11 |     static {
>>>   12 |         System.loadLibrary(soName);
      13 |     }
      14 | }
```

Severity rationale: Medium: a planted .so executes with the app's privileges, but requires the attacker to write to that location.

Confidence rationale: Low: the call shape is suspicious, but whether the variable/path is actually attacker-influenced needs data-flow review.

Remediation

Load native libraries only with System.loadLibrary("<literal-name>") so the loader resolves them from the APK's trusted lib directory. Never System.load() a path under external storage or other writable directories; verify checksums if a path must be dynamic.

13. Sensitive data written to external storage

MEDIUM CONFIDENCE: MEDIUM STATIC

94b22f97-0010 · rule BS-S05 v1.0.0 · engine: custom · storage

Location	jadx\sources\jakhar\aseem\diva\InsecureDataStorage4Activity.java : lines 24
OWASP	M9
CWE	CWE-312

Description

Found 1 external-storage call(s) adjacent to sensitive keywords.

Evidence

```
       22 |         EditText usr = (EditText) findViewById(R.id.ids4Usr);
       23 |         EditText pwd = (EditText) findViewById(R.id.ids4Pwd);
>>>    24 |         File sdir = Environment.getExternalStorageDirectory();
       25 |         try {
       26 |             File uinfo = new File(sdir.getAbsolutePath() + "/.uinfo.txt");
```

Severity rationale: External storage is broadly readable; sensitive data written there is exposed to other apps.

Confidence rationale: The external-storage call is real, but the nearby keyword is a heuristic link rather than proven data flow.

Remediation

Keep sensitive data in internal storage (getFilesDir) or EncryptedSharedPreferences/Android Keystore. Only non-sensitive data belongs on external storage.

14. Sensitive data in SharedPreferences without EncryptedSharedPreferences

MEDIUM CONFIDENCE: LOW HEURISTIC

94b22f97-0011 · rule BS-S06 v1.0.0 · engine: custom · storage

Location	jadx\sources\jakhar\aseem\diva\InsecureDataStorage1Activity.java : lines 25
OWASP	M9
CWE	CWE-312

Description

Found 1 sensitive putString() call(s) with no EncryptedSharedPreferences usage in the app.

Evidence

```
       23 |         EditText pwd = (EditText) findViewById(R.id.ids1Pwd);
       24 |         spedit.putString("user", usr.getText().toString());
>>>    25 |         spedit.putString("password", pwd.getText().toString());
       26 |         spedit.commit();
       27 |         Toast.makeText(this, "3rd party credentials saved successfully!", 0).show();
```

Severity rationale: If the stored values are real secrets, they sit in plaintext XML readable on rooted devices.

Confidence rationale: The preference keys look sensitive and no encrypted preferences API is used, but the actual runtime values cannot be confirmed statically.

Remediation

Store sensitive values with EncryptedSharedPreferences (AndroidX Security) backed by Android Keystore, or move them server-side. Avoid persisting passwords at all; persist short-lived tokens.

Also detected by: custom

15. Sensitive information written to logs

MEDIUM CONFIDENCE: MEDIUM STATIC

94b22f97-0012 · rule BS-S10 v1.0.0 · engine: custom · storage

Location	jadx\sources\jakhar\aseem\diva\LogActivity.java : lines 23
OWASP	M9
CWE	CWE-532

Description

Found 4 logging call(s) with sensitive keywords.

Evidence

```
       21 |             processCC(cctxt.getText().toString());
       22 |         } catch (RuntimeException e) {
>>>    23 |             Log.e("diva-log", "Error while processing transaction with credit card: " +
cctxt.getText().toString());
       24 |             Toast.makeText(this, "An error occured. Please try again later", 0).show();
       25 |         }

      504 |             Log.d(TAG, "  mServiceCallbacks=" + this.mServiceCallbacks);
      505 |             Log.d(TAG, "  mRootId=" + this.mRootId);
>>>   506 |             Log.d(TAG, "  mMediaSessionToken=" + this.mMediaSessionToken);
      507 |         }
      508 |

      581 |                 return this.mBinder.getLaunchPendingIntent();
      582 |             } catch (RemoteException e) {
>>>   583 |                 Log.e(MediaControllerCompat.TAG, "Dead object in getSessionActivity. " + e);
      584 |                 return null;
      585 |             }

      111 |                 mediaButtonEventReceiver = new ComponentName(resolveInfo.activityInfo.packageName,
resolveInfo.activityInfo.name);
      112 |             } else if (resolveInfos.size() > 1) {
>>>   113 |                 Log.w(TAG, "More than one BroadcastReceiver that handles
android.intent.action.MEDIA_BUTTON was found, using null. Provide a specific ComponentName to use as this session's
media button receiver");
      114 |             }
      115 |         }
```

Severity rationale: Secrets in logcat are exposed to other apps, ADB users, and bug reports.

Confidence rationale: The keyword appears in the logged statement itself, but whether the actual secret value (vs. a label) is logged needs a look at the surrounding code.

Remediation

Never log secrets, tokens, or credentials. Log redacted or hashed identifiers instead, strip logging in release builds (BuildConfig.DEBUG guards / ProGuard), and audit existing calls.

16. WebView JavaScript enabled

MEDIUM CONFIDENCE: HIGH AI-CORROBORATED

94b22f97-0016 · rule `BS-W01` v1.0.0 · engine: custom · webview

Location	`jadx\sources\jakhar\aseem\diva\InputValidation2URISchemeActivity.java` : lines 18
OWASP	M8
CWE	CWE-749

Description

WebView.setJavaScriptEnabled(true) found at 1 call site(s).

Evidence

```
       16 |         WebView wview = (WebView) findViewById(R.id.ivi2wview);
       17 |         WebSettings wset = wview.getSettings();
>>>    18 |         wset.setJavaScriptEnabled(true);
       19 |     }
       20 |
```

Severity rationale: MEDIUM: JavaScript in a WebView materially widens attack surface (XSS, malicious ads/pages) but is a legitimate feature on its own.

Confidence rationale: HIGH: setJavaScriptEnabled(true) is a direct, unambiguous API call.

Remediation

Disable JavaScript unless the WebView genuinely needs it. If it is required, only load trusted HTTPS content, validate every URL in shouldOverrideUrlLoading, and never combine JavaScript with an addJavascriptInterface bridge that reaches native functionality.

Also detected by: ai

17. WebView loads cleartext HTTP or dynamically built URLs

MEDIUM CONFIDENCE: LOW HEURISTIC

94b22f97-0017 · rule BS-W04 v1.0.0 · engine: custom · webview

Location	jadx\sources\jakhar\aseem\diva\InputValidation2URISchemeActivity.java : lines 24
OWASP	M4

Description

WebView loads dynamically constructed URLs (1 call site(s)); destinations need manual review.

Evidence

```
       22 |         EditText uriText = (EditText) findViewById(R.id.ivi2uri);
       23 |         WebView wview = (WebView) findViewById(R.id.ivi2wview);
>>>    24 |         wview.loadUrl(uriText.getText().toString());
       25 |     }
       26 | }
```

Severity rationale: MEDIUM: a dynamically built URL is a prerequisite for open redirect / intent-driven WebView abuse if the input is attacker-controlled.

Confidence rationale: LOW: concatenation of a variable was observed, but the source of the variable (trusted constant vs. user/intent input) and exploitability were not established.

Remediation

Load only https:// URLs in WebViews. Build URLs from allow-listed constants, validate them in shouldOverrideUrlLoading, and never pass raw intent data or user input into loadUrl.

Also detected by: custom

18. SQL Injection Vulnerability

MEDIUM CONFIDENCE: LOW AI-EVIDENCE-SUPPORTED

94b22f97-0024 · rule ai:llm-review v · engine: ai

Location	jadx\sources\jakhar\aseem\diva\NotesProvider.java : lines 48–63

Description

The `onCreate` method in the `DBHelper` class directly inserts hardcoded SQL statements into the database, which can lead to SQL injection if the input is not properly sanitized.

Evidence

```
db.execSQL("INSERT INTO notes(title,note) VALUES ('office', '10 Meetings. 5 Calls. Lunch with CEO');");
```

19. Hardcoded Sensitive Information

MEDIUM CONFIDENCE: HIGH AI-EVIDENCE-SUPPORTED

94b22f97-0025 · rule `ai:llm-review` v · engine: ai

Location	`jadx\sources\jakhar\aseem\diva\APICreds2Activity.java` : lines 14–15
OWASP	M1
CWE	CWE-89

Description

The API key, username, and password are hardcoded in the code. Why this is unsafe: Hardcoding sensitive information exposes it to anyone who can access the source code. Preconditions: The application is running and the hardcoded credentials are visible.

Evidence

```
apicview.setText("TVEETER API Key: secrettveeterapikey\nAPI User name: diva2\nAPI Password: p@ssword2");
```

Impact

This can lead to unauthorized access to the API and potential data breaches.

Remediation

Remove hardcoded credentials and use secure methods to store and retrieve them, such as environment variables or secure vaults.

20. Insecure Data Storage

MEDIUM CONFIDENCE: HIGH AI-UNVERIFIED

94b22f97-0026 · rule `ai:llm-review` v · engine: ai

Location	`jadx\sources\jakhar\aseem\diva\InsecureDataStorage1Activity.java`
OWASP	M4
CWE	CWE-89

Description

The application stores user credentials (username and password) in SharedPreferences without encryption, making them vulnerable to unauthorized access. Why this is unsafe: SharedPreferences stores data in plain text, which can be easily read by any application with access to the device. Preconditions: The device must be rooted or have a custom ROM that allows access to the SharedPreferences files.

Evidence

```
spedit.putString("user", usr.getText().toString());
spedit.putString("password", pwd.getText().toString());
spedit.commit();
```

Impact

An attacker can steal user credentials and potentially gain unauthorized access to the user's account.

Remediation

Encrypt the data before storing it in SharedPreferences. Use Android's Keystore System or a third-party encryption library.

21. SQL Injection Vulnerability

MEDIUM CONFIDENCE: HIGH AI-EVIDENCE-SUPPORTED

94b22f97-0027 · rule `ai:llm-review` v · engine: ai

Location	`jadx\sources\jakhar\aseem\diva\InsecureDataStorage2Activity.java` : lines 24
OWASP	M4
CWE	CWE-89

Description

The code is vulnerable to SQL injection because it directly concatenates user input into SQL queries without proper sanitization or parameterized queries. Why this is unsafe: User input is directly concatenated into the SQL query, allowing attackers to manipulate the query and potentially execute arbitrary SQL code. Preconditions: User input must be controlled and sanitized.

Evidence

```
this.mDB.execSQL("INSERT INTO myuser VALUES ('" + usr.getText().toString() + "', '" + pwd.getText().toString() +
"');");
```

Impact

An attacker could potentially retrieve, modify, or delete data from the database.

Remediation

Use parameterized queries or prepared statements to prevent SQL injection.

22. SQL Injection Vulnerability

MEDIUM CONFIDENCE: HIGH AI-EVIDENCE-SUPPORTED

94b22f97-0028 · rule `ai:llm-review` v · engine: ai

Location	`jadx\sources\jakhar\aseem\diva\SQLInjectionActivity.java` : lines 26
OWASP	M4
CWE	CWE-89

Description

The application is vulnerable to SQL Injection due to the direct concatenation of user input into SQL queries. Why this is unsafe: User input is directly concatenated into the SQL query, allowing an attacker to manipulate the query and potentially execute arbitrary SQL code. Preconditions: The application must be accessed by an attacker who can input data into the search field.

Evidence

```
Cursor cr = this.mDB.rawQuery("SELECT * FROM sqliuser WHERE user = '" + srchtxt.getText().toString() + "'", null);
```

Impact

An attacker could exploit this vulnerability to retrieve, modify, or delete data from the database.

Remediation

Use parameterized queries or prepared statements to prevent SQL Injection.

23. Hardcoded Sensitive Information

MEDIUM CONFIDENCE: HIGH AI-EVIDENCE-SUPPORTED

94b22f97-0029 · rule `ai:llm-review` v · engine: ai

Location	`jadx\sources\jakhar\aseem\diva\APICredsActivity.java` : lines 10
OWASP	M1
CWE	CWE-89

Description

The API key, username, and password are hardcoded in the source code. Why this is unsafe: Hardcoding sensitive information directly in the source code makes it easily accessible to anyone with access to the codebase. Preconditions: The application is deployed and accessible to users.

Evidence

```
apicview.setText("API Key: 123secretapikey123\nAPI User name: diva\nAPI Password: p@ssword");
```

Impact

This can lead to unauthorized access to the API, potential data breaches, and financial losses.

Remediation

Remove hardcoded sensitive information and store it securely using environment variables, secure vaults, or secure configuration management tools.

24. Hardcoded Secret Key

MEDIUM CONFIDENCE: HIGH AI-EVIDENCE-SUPPORTED

94b22f97-0030 · rule `ai:llm-review` v · engine: ai

Location	`jadx\sources\jakhar\aseem\diva\HardcodeActivity.java` : lines 14
OWASP	M1
CWE	CWE-89

Description

The application contains a hardcoded secret key that is used to authenticate users. This key should be stored securely and not hardcoded in the source code. Why this is unsafe: Hardcoding sensitive information such as secret keys in the source code makes it easily accessible to anyone who can access the source code. This can lead to unauthorized access to the application. Preconditions: The application is deployed in a production environment.

Evidence

```
hckey.getText().toString().equals("vendorsecretkey")
```

Impact

If an attacker gains access to the hardcoded secret key, they can bypass authentication and gain unauthorized access to the application.

Remediation

Store the secret key securely using a secure key management system. Avoid hardcoding sensitive information in the source code.

25. Insecure Storage of Sensitive Data

MEDIUM CONFIDENCE: HIGH AI-EVIDENCE-SUPPORTED

94b22f97-0031 · rule `ai:llm-review` v · engine: ai

Location	`jadx\sources\jakhar\aseem\diva\AccessControl3Activity.java` : lines 17–21
OWASP	M1
CWE	CWE-312

Description

The PIN is being stored in SharedPreferences without encryption, which can be easily accessed by other apps on the device. Why this is unsafe: SharedPreferences stores data in plain text, making it vulnerable to unauthorized access. Preconditions: The device must have multiple apps installed, and the attacker must have access to the device.

Evidence

```
spedit.putString(getString(R.string.pkey), pin);
```

Impact

An attacker could potentially retrieve the PIN and gain access to protected data.

Remediation

Use a secure storage mechanism like Keystore or EncryptedSharedPreferences to store sensitive data.

26. Insecure Data Storage

MEDIUM CONFIDENCE: HIGH AI-UNVERIFIED

94b22f97-0032 · rule `ai:llm-review` v · engine: ai

Location	`jadx\sources\jakhar\aseem\diva\InsecureDataStorage4Activity.java`
OWASP	M4
CWE	CWE-89

Description

The application is storing user credentials in a file with read and write permissions, which can be accessed by any application on the device. Why this is unsafe: The file is stored in the external storage with read and write permissions, making it accessible to other applications on the device. Preconditions: The device must have external storage enabled and accessible.

Evidence

```
File sdir = Environment.getExternalStorageDirectory();
try {
    File uinfo = new File(sdir.getAbsolutePath() + "/.uinfo.txt");
    uinfo.setReadable(true);
    uinfo.setWritable(true);
    FileWriter fw = new FileWriter(uinfo);
    fw.write(usr.getText().toString() + ":" + pwd.getText().toString() + "\n");
    fw.close();
    Toast.makeText(this, "3rd party credentials saved successfully!", 0).show();
} catch (Exception e) {
    Toast.makeText(this, "File error occurred", 0).show();
    Log.d("Diva", "File error: " + e.getMessage());
}
```

Impact

Sensitive data like user credentials can be accessed by other applications, leading to potential unauthorized access and data breaches.

Remediation

Store sensitive data in a secure location, such as the internal storage or a secure vault, and restrict access to it.

27. Dangerous permissions declared

LOW CONFIDENCE: HIGH STATIC

94b22f97-0004 · rule BS-M05 v1.0.0 · engine: custom · manifest

Location	AndroidManifest.xml
OWASP	M6

Description

2 dangerous permission(s) across 1 sensitive group(s): Storage: READ_EXTERNAL_STORAGE, WRITE_EXTERNAL_STORAGE.

Evidence

```
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />

<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />
```

Severity rationale: LOW: SMS, call-log, and precise/background location permissions are MEDIUM because they enable surveillance-grade data collection; other dangerous groups (contacts, camera, microphone, storage, phone, calendar, sensors) are LOW as privacy-relevant but less directly exfiltrative on their own.

Confidence rationale: HIGH: permissions are literal <uses-permission> entries parsed from the manifest; Android's dangerous classification is documented platform behavior.

Remediation

Request only permissions the app genuinely needs; prefer scoped alternatives (photo picker, approximate location, one-time permissions). Justify each dangerous permission in the Play Data safety section and handle denial gracefully.

28. Sensitive data in temporary files

LOW CONFIDENCE: MEDIUM STATIC

94b22f97-0013 · rule BS-S19 v1.0.0 · engine: custom · storage

Location	`jadx\sources\jakhar\aseem\diva\InsecureDataStorage3Activity.java` : lines 25
OWASP	M9
CWE	CWE-377

Description

Found 1 temporary-file call(s) linked to sensitive data.

Evidence

```
       23 |         File ddir = new File(getApplicationInfo().dataDir);
       24 |         try {
>>>    25 |             File uinfo = File.createTempFile("uinfo", "tmp", ddir);
       26 |             uinfo.setReadable(true);
       27 |             uinfo.setWritable(true);
```

Severity rationale: Temp files are plaintext and may linger; low impact unless highly sensitive data is written.

Confidence rationale: The temp-file creation is real and the sensitive link is in the file name or immediately adjacent code.

Remediation

Avoid writing secrets to temp files. If unavoidable, use the app-private cache directory, set restrictive permissions, and delete the file immediately after use.

29. No network security configuration referenced

INFORMATIONAL CONFIDENCE: HIGH STATIC

94b22f97-0007 · rule BS-M15 v1.0.0 · engine: custom · manifest

Location	`AndroidManifest.xml`
OWASP	M5

Description

No android:networkSecurityConfig is referenced in the manifest. Informational: no pinning, per-domain TLS policy, or cleartext exceptions are declared.

Evidence

```
android:networkSecurityConfig not declared
```

Severity rationale: INFORMATIONAL: the absence of a network security config is inventory metadata, not a vulnerability; it only means no declarative network controls exist.

Confidence rationale: HIGH: the attribute is either present on the application tag or it is not; absence is directly observed.

Remediation

No action required by this finding alone. Consider adding a network security config to declare certificate pinning, restrict cleartext traffic per-domain, and distrust user CAs in release builds.

30. Certificate pinning presence check

INFORMATIONAL CONFIDENCE: MEDIUM STATIC

94b22f97-0014 · rule BS-N08 v1.0.0 · engine: custom · network

OWASP	M5

Description

No certificate pinning detected; assess threat model. Absence alone is not a vulnerability.

Severity rationale: Informational: pinning is a defense-in-depth choice, not a vulnerability by itself.

Confidence rationale: Heuristic keyword search; custom or native pinning may be missed, hence medium confidence.

Remediation

If your threat model includes hostile networks, add certificate pinning (e.g. OkHttp CertificatePinner or <pin-set> in the network security config) with a backup pin and a rotation plan.

31. INTERNET permission without a network security configuration

INFORMATIONAL CONFIDENCE: HIGH STATIC

94b22f97-0015 · rule BS-N20 v1.0.0 · engine: custom · network

OWASP	M5

Description

App uses INTERNET but declares no network security configuration.

Severity rationale: Informational: absence of a config is not itself a vulnerability; it limits declarative transport-policy review.

Confidence rationale: Manifest declares INTERNET and has no android:networkSecurityConfig attribute.

Remediation

Add an explicit network security configuration declaring cleartextTrafficPermitted="false", appropriate trust anchors, and pinning where the threat model warrants it.

32. Native library inventory

INFORMATIONAL CONFIDENCE: HIGH STATIC

94b22f97-0022 · rule BS-P09 v1.0.0 · engine: custom · platform

Location	`lib/arm64-v8a/libdivajni.so`
OWASP	M7

Description

APK ships 7 native librar(ies): lib/arm64-v8a/libdivajni.so, lib/armeabi-v7a/libdivajni.so, lib/armeabi/libdivajni.so, lib/mips/libdivajni.so, lib/mips64/libdivajni.so, lib/x86/libdivajni.so, lib/x86_64/libdivajni.so

Severity rationale: Informational: inventory only, not a vulnerability.

Confidence rationale: High: the list comes directly from the APK's lib/ entries.

Remediation

No action required from this inventory alone. Review the listed libraries for known-vulnerable versions and confirm they are built with modern hardening (PIE, RELRO, stack canaries).

Limitations and exclusions

- Static analysis only - no runtime, network, or backend testing was performed.
- Findings marked *heuristic* or *ai-unverified* require manual confirmation.
- Absence of a finding is not proof of absence of the vulnerability.